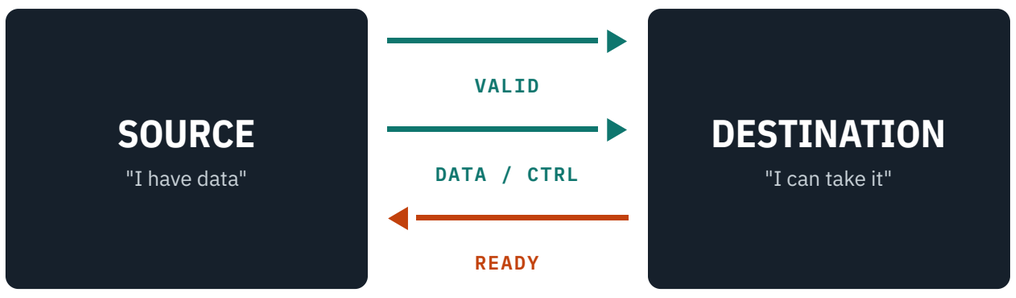
DV INTERVIEW · GLANCE SHEET 4

The VALID–READY Handshake, In Depth

One mechanism, used identically on all five channels. Get this right and most AXI questions answer themselves.

1 HIGH
⇄ handshake edge
✗ protocol violation

1 The idea in one picture



Transfer = VALID && READY at a rising ACLK edge. Nothing else moves data.

Channel	Source (VALID)	Destination (READY)
AW, W, AR	Master	Slave
B, R	Slave	Master

2 The five rules

#	Rule	Why
R1	Source **must not wait for READY** before asserting VALID.	If both sides wait for each other → deadlock.
R2	Once VALID is HIGH it **stays HIGH until the handshake**.	Destination may sample any later cycle; data can't vanish.
R3	While VALID is HIGH and not yet accepted, **payload stays stable**.	Destination must get exactly what was offered.
R4	Destination **may wait for VALID** before asserting READY.	Lets slaves be simple and save power.
R5	READY **may be asserted early** and **may be de-asserted** before VALID arrives.	READY is only a "can accept now" hint until VALID shows up.

Asymmetry to remember: VALID is a *commitment* (can't take it back). READY is an *offer* (can withdraw it while VALID is low).

3 The three legal cases

CASE A · VALID BEFORE READY

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	1	⇄	0	0
READY	0	0	0	⇄	0	0
DATA	–	D	D	D	–	–

Source waits with D held stable (R2, R3). Typical when the slave is busy.

CASE B · READY BEFORE VALID

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	0	⇄	0	0
READY	0	1	1	⇄	0	0
DATA	–	–	–	D	–	–

Destination is idle and waiting. Transfer happens the first cycle VALID rises — zero wait.

CASE C · BOTH IN THE SAME CYCLE

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	⇄	0	0	0
READY	0	0	⇄	0	0	0
DATA	–	–	D	–	–	–

Both assert together → transfer in that same cycle. One beat per cycle is the max rate.

Back-to-back streaming: keep VALID and READY both HIGH and change DATA every cycle → one transfer per clock (100% throughput). Every cycle with VALID && !READY or READY && !VALID is a bubble.

✗ Illegal cases — the ones interviewers draw on the board

VIOLATION 1 · VALID DROPS EARLY (BREAKS R2)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	0 ✗	1	0	0
READY	0	0	0	0	0	0

Source "took back" its offer at T3 without a handshake.

VIOLATION 2 · DATA CHANGES WHILE STALLED (BREAKS R3)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	1	⇄	0	0
READY	0	0	0	⇄	0	0
DATA	–	D0	D1 ✗	D1	–	–

D0 was offered but D1 got accepted — D0 silently lost.

VIOLATION 3 · SOURCE WAITS FOR READY (BREAKS R1)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	0	0	0	0 …
READY	0	0	0	0	0	0 …

Source waits for READY, slave waits for VALID (allowed by R4) → **deadlock forever**.

Not a violation: READY going HIGH then LOW while VALID is LOW (R5). Also legal: VALID HIGH for many cycles with READY LOW — that's just back-pressure.

4 Cross-channel dependencies

READ

`ARVALID` + `ARREADY` must both happen before `RVALID`

ARREADY itself may be asserted before or after ARVALID.

WRITE (AXI4)

`AW handshake` + `W handshake with WLAST` must both happen before `BVALID`

- Slave may wait for AWVALID, WVALID or both before raising AWREADY.
- Slave may wait for AWVALID, WVALID or both before raising WREADY.
- Master must **not** wait for AWREADY before WVALID, nor for WREADY before AWVALID.
- AXI3 was looser: BVALID only had to follow WLAST, not AW.

Mental model: a slave can always say "I'll wait for you". A master can never say "you go first".

5 Timing & implementation notes

- Everything is sampled on the **rising edge** of ACLK.
- **No combinational path** from inputs to outputs on an interface — e.g. READY must not be a pure function of VALID through logic in the same cycle at the interface boundary.
- READY travels **backward**; long READY paths hurt timing → insert a **register slice**.
- A full-throughput register slice needs a **2-entry skid buffer**: when READY is registered, one extra beat may arrive after downstream stalls, so it must be stored.
- During reset, all VALIDs LOW; first VALID only after ARESETn HIGH.
- Good practice: masters keep `BREADY` / `RREADY` HIGH unless they truly can't accept.

Throughput math: efficiency = handshakes ÷ cycles. 4 beats over 6 cycles = 67%; 2 bubbles came from back-pressure or idle VALID.

6 Handshake assertions (SVA)

R2 — VALID HELD UNTIL READY

```
property p_valid_hold;
  @(posedge ACLK) disable iff (!ARESETn)
  AWVALID && !AWREADY |=> AWVALID;
endproperty
```

R3 — PAYLOAD STABLE WHILE STALLED

```
property p_payload_stable;
  @(posedge ACLK) disable iff (!ARESETn)
  AWVALID && !AWREADY |=>
    $stable({AWADDR, AWLEN, AWSIZE, AWBURST, AWID});
endproperty
```

RESET & X CHECKS

```
a_rst_low : assert property (@(posedge ACLK) !ARESETn |-> !AWVALID);
a_no_x    : assert property (@(posedge ACLK) disable iff (!ARESETn)
                             !$isunknown(AWVALID));
```

Copy the same set for W, AR (master side) and B, R (slave side). Add a liveness/timeout check: VALID must eventually see READY within N cycles.

? Handshake interview Q&A

Question	Crisp answer
When exactly does a transfer happen?	Rising edge where VALID and READY are both HIGH.
Can VALID depend on READY?	No — that's the deadlock rule (R1).
Can READY depend on VALID?	Yes — destination may wait for VALID.
Can READY drop without a transfer?	Yes, if VALID was LOW.
Can VALID drop without a transfer?	No — protocol violation.
Max throughput of one channel?	One transfer per clock (VALID & READY both held HIGH).
Why a skid buffer in a register slice?	Registered READY lags by a cycle; the in-flight beat needs a slot.
How do you test the handshake in a TB?	Randomize VALID/READY delays incl. 0; cover cases A/B/C; SVA for hold/stable/reset.
Bug: slave waits for WVALID before AWREADY, master waits for AWREADY before WVALID?	Deadlock; the master is the one breaking the rule.